INT161 · Basic Backend Development

Introduction

เว็บคุยกันอย่างไร: HTTP request และ response, SPA กับ JSON และทำไมเราเขียน backend ด้วย Node.js

Course and Exam | Web Application and HTTP | MPA, SPA and JSON | REST and SOAP

What is Node.js | Your First Server | อภิธานศัพท์

01

Course and Exam

คุณจะเขียน **back end**: โปรแกรมที่คุยกับ **front end** ผ่าน **HTTP** และเก็บข้อมูลใน database

2nd exam, theory: 30%

Multiple choice, single answer

2nd exam, practice: 20%

Write a program that solves the problem

3rd exam, theory: 30%

Multiple choice, single answer

3rd exam, practice: 20%

Write a program that solves the problem

- ข้ามรอบแรก วิชานี้สอบรอบ 2 และ 3
- สอบปฏิบัติทำบนเครื่อง อาจารย์อยากให้มี **server** ตรวจโปรแกรมแล้วบอกผ่านหรือไม่ผ่าน
- หลักให้คะแนนข้อแรก: โปรแกรมต้องทำงานได้ตามโจทย์ก่อน
- ฝึกทำแบบฝึกหัดออนไลน์ตั้งแต่เนิ่น ๆ login ไว้เพื่อให้เว็บจำว่าทำถึงไหนแล้ว

> **ในแผนการสอน**
>
> สัปดาห์ถัดไปในแผนการสอน: ORM และ Prisma, validation, filtering และ sorting, JWT, authentication และโปรเจกต์รวมที่ deploy ด้วย container โน้ตนี้ครอบคลุมคลาสที่สอนแล้ว

02

Web Application and HTTP

อาจารย์เปรียบกับการขับรถ ก่อนจะขับเก่งต้องรู้เรื่องถนน ถนนของ web app คือ World Wide Web

- ต้องมีสองโปรแกรม: **browser** ฝั่งหนึ่ง และ **web server** อีกฝั่ง
- browser ไม่ใช่ **client** อย่างเดียว เครื่องมืออย่าง **curl** ก็เป็น client
- **IP address** หาเครื่อง **port** หาโปรแกรมบนเครื่อง ถ้าไม่ใส่ port browser ใช้ 80 เครื่องเดียวรันหลาย server ได้ถ้าคนละ port
- ไม่มีใครจำ IP address ได้ **DNS** จึงแปลงชื่อเป็น IP

ตัวข้อความเป็นข้อความธรรมดา คลิกแต่ละบรรทัดเพื่อดูว่าทำหน้าที่อะไร

Request | Response

Client → Server

```
GET /user HTTP/1.1
Host: localhost:3000
Accept: application/json

{ "name": "Somchai" }
```

บรรทัดแรก: method, **URL** และ version method บอกว่าจะทำอะไร

Stateless

ตัวอย่างของอาจารย์: สิ้นเดือนโทรไปขอเงินแม่ 3000 แม่โอนมาแล้วจบ เดือนหน้าต้องขอใหม่ แม่ไม่ได้เช็คว่าเรายังอยู่ไหม server ก็ทำกับ **request** แบบเดียวกัน ทุก request จึงต้องพกสิ่งที่ต้องใช้มาเอง

บรรทัดแรกของ **response** สำคัญที่สุดตอนเขียนโปรแกรม เลือกสถานการณ์ดู

เจอข้อมูล | บันทึกแถวใหม่ | ไม่เจอข้อมูล | ยังไม่ login | login แล้วแต่ไม่มีสิทธิ์ | code ของเราพัง

ส่งกลับ

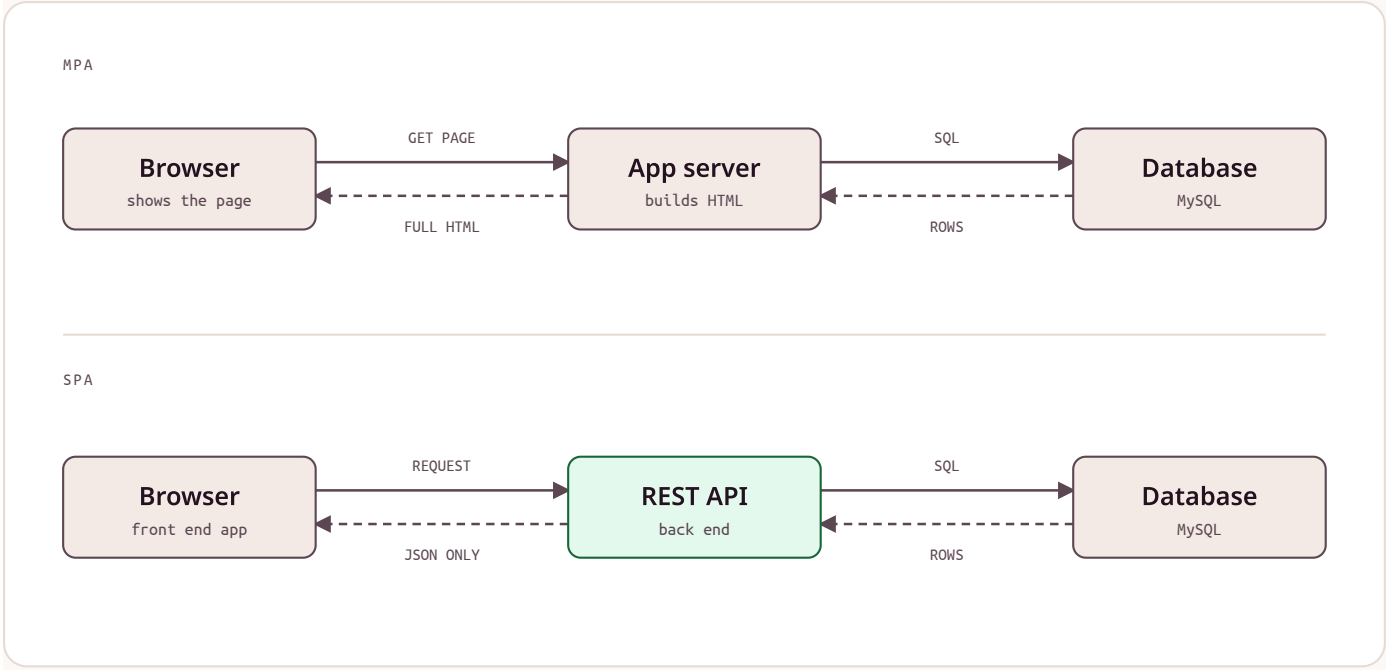
`200 OK`

request สำเร็จและ **response** มีข้อมูล

- front end อ่าน code ถ้าไม่ใช่ 200 ต้องแปลง code เป็นข้อความให้ผู้ใช้ เช่น "คุณไม่มีสิทธิ์"
- back end ต้องรู้ว่า **URL** ที่เข้ามาให้ฟังก์ชันไหนตอบ นั่นคือ **URL mapping** ทุกเครื่องมือมี ส่วนใหญ่เรียกว่า **router**

03

MPA, SPA and JSON



- front end กับ back end เป็นสองโปรเจกต์แยกกัน front end ถูกโหลดมาที่ browser ส่วน back end อยู่บน server
- โทรศัพท์ทำให้เกิดการเปลี่ยนนี้ โทรศัพท์ไม่ควร render หน้า HTML ใหญ่ ๆ และฐานข้อมูลนักศึกษาก็อยู่บนโทรศัพท์ไม่ได้ แอปเป็น front end แล้วเรียก **API**

JSON เป็นข้อความธรรมดา object ขึ้นต้นด้วย { ชื่อ field เป็น string ในเครื่องหมายคำพูดคู่ ตามด้วย : แล้วค่า

```JSON
{
  "name": "John Doe",
  "age": 21,
  "subjects": ["INT161", "INT134"]
}
```

- ค่าเป็นได้ทั้ง string, number, object และ array
- คั่น field ด้วยจุลภาค หลัง field สุดท้ายปิดด้วย }

04

REST and SOAP

web service คือ back end ที่สร้างให้โปรแกรมอื่นใช้ ตัวอย่างอาจารย์: eBay ขายของ PayPal โอนเงิน

- eBay อยากรู้ว่าผู้ซื้อจ่ายแล้วหรือยัง ถ้าเปิด database ของ PayPal ให้ eBay จะเห็นทุกตาราง
- PayPal จึงสร้าง web service eBay ส่งเลข reference ไปและได้กลับมาแค่ transaction นั้น
- ไอเดียเดียวกับบริษัทขนส่ง: eBay ส่ง tracking number แล้วได้สถานะกลับมา

SOAP

ข้อความ XML ที่อธิบายด้วย WSDL หนัก แต่รองรับลายเซ็นดิจิทัล หน่วยงานรัฐยังใช้อยู่

REST

แนวทางออกแบบ ไม่ใช่ **protocol** ใช้สิ่งที่ HTTP มีอยู่แล้วและตอบเป็น JSON ขนาดเล็ก คิดโดย Roy Fielding

> **วิชานี้อยู่ตรงไหน**
> เราเขียน **REST** API ซึ่งเริ่มจาก web service ระหว่าง server แล้วขยายมาเป็น back end ของแอป front end

05

What is Node.js

Node.js ไม่ใช่ภาษา เป็น **runtime** ที่รัน JavaScript บน server ทำให้ใช้ภาษาเดียวได้ทั้งสองฝั่ง

Compiled

C, C++, Go

แปลทั้งโปรแกรมเป็น machine code ก่อน รันเร็ว ผูกกับ platform เดียว source เป็นความลับ

Interpreted

JavaScript on Node.js

อ่านทีละ statement รันได้ทุกที่ที่มี **interpreter** ผิดที่บรรทัด 5 จะรู้ตอนรันถึงบรรทัด 5

Hybrid

Java, Kotlin, C#

compile เป็น bytecode ครั้งเดียว แล้ว virtual machine รันบนทุก platform เป็นทางสายกลาง

- Node.js อยู่กลุ่ม interpreter อีก runtime หนึ่งคือ Bun ทำงานเดียวกันแต่คนละทีม code ไม่ได้การันตีว่าเหมือนกันเป๊ะ
- รันแบบ **async** บน thread เดียว เหมือนสั่งข้าวผัดร้านคิวยาว แทนที่จะยืนรอ ฝาก **callback** ไว้แล้วไปสั่งอย่างอื่น keyword คือ async กับ **await**

install

```
# install nvm, then Node (slide uses version 22)
$ nvm install 22
$ node -v
v22.14.0
$ npm -v
10.9.2
$ node app.js
```

- **nvm** ให้เครื่องเดียวมีหลาย **Node** version **npm** มากับ Node และติดตั้ง package เพิ่ม
- เลือก **LTS** ถ้าไม่ใช่ LTS อาจเลิกแก้ช่องโหว่

06

Your First Server

สามขั้นตอน: import http **module** สร้าง server พร้อมฟังก์ชันที่จัดการทุก request แล้ว listen ที่ port

my-first-app.js

```
const http = require('node:http');
const listener = function (request, response) {
  response.writeHead(200, { 'Content-Type': 'text/html' });
  response.end('<h2>Hello World</h2>');
};
const server = http.createServer(listener);
server.listen(3000);
console.log('Server running at http://127.0.0.1:3000/');
```

- createServer แค่บอกว่าจะทำอะไรเมื่อมี request ยังไม่ทำงานจนกว่าจะเรียก listen
- **header** ระบุ Content-Type ถ้าเป็น text/plain browser จะพิมพ์ HTML เป็นข้อความ ถ้าเป็น text/html จะ render

แยกตาม request.url เพื่อตอบแต่ละ URL ด้วย **status code** ที่ต่างกัน

my-third-app.js

```
const http = require('node:http');
const listener = function (request, response) {
  if (request.url === '/user') {
    response.writeHead(200, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify({ name: 'John Doe' }));
  } else if (request.url === '/') {
    response.writeHead(200);
    response.end('Hello World');
  } else {
    response.writeHead(404);
    response.end('Error: Page Not Found');
  }
};
http.createServer(listener).listen(3000);
```

> ⚠ **โจทย์ฝึก**
> ส่งตัวเลขใน URL เช่น /?x=5 อ่านจาก request แล้วตอบเป็นสูตรคูณแม่ 5 ข้อมูลอยู่ใน request โปรแกรมจึงต้องอ่านจาก request

07

อภิธานศัพท์

API
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

async
: เริ่มงานที่ช้า แล้วทำงานอื่นต่อ และถูกเรียกกลับเมื่อเสร็จ แทนที่จะรอ

await
: หยุด async function นี้จนกว่า Promise จะเสร็จ ใช้ได้เฉพาะใน async function

back end
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

browser
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

callback
: ฟังก์ชันที่ส่งไปให้เรียกทีหลังเมื่องานเสร็จ

client
: ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

curl
: client แบบ command line ส่ง HTTP method ใดก็ได้ ใช้ทดสอบ POST และ PUT ได้ ใส่ -i เพื่อดู header ของ response

DNS
: Domain Name Service: แปลงชื่ออย่าง sit.kmutt.ac.th เป็น IP address ของเครื่อง

front end
: ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

header
: ข้อมูลเสริมของข้อความ ส่งเป็นบรรทัด name: value เช่น Content-Type

HTTP
: HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

interpreter
: อ่านและรัน source ทีละ statement ไม่มีขั้นตอน build แต่ต้องมี source บนทุกเครื่อง

IP address
: เลขที่ระบุเครื่องหนึ่งเครื่องบน network ไม่มีสองเครื่องที่ซ้ำกัน

JSON

JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

LTS

Long Term Support: Node version ที่ได้รับการแก้ช่องโหว่ต่อเนื่องหลายปี ควรเลือกใช้

module

ไฟล์ code ที่โหลดเข้าไฟล์อื่นด้วย import

Node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

Node.js

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

npm

Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

nvm

Node Version Manager: ติดตั้งหลาย Node version ไว้ด้วยกันและสลับได้

port

เลขที่บอกว่าโปรแกรมไหนบนเครื่องควรได้รับข้อความ web server ใช้ 80 เป็นค่าเริ่มต้น

protocol

กฎที่ตกลงกันว่าสองโปรแกรมจะคุยกันอย่างไร

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

response

สิ่งที่ server ส่งกลับ: status code, header และ body

REST

แนวทางออกแบบ web API: resource ตั้งชื่อด้วย URL และเลือก action ด้วย HTTP method

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

runtime

สภาพแวดล้อมที่รัน code ของเรา Node.js เป็น runtime ของ JavaScript ไม่ใช่ภาษา

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

Stateless

server ไม่จำอะไรระหว่าง request แต่ละ request ต้องพกข้อมูลที่ต้องใช้มาครบ

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

URL

ที่อยู่ของ resource ประกอบด้วย protocol, host, port และ path เช่น http://localhost:3000/user

URL mapping

การเลือกว่าฟังก์ชันไหนจัดการ URL ไหน framework เรียกว่า router

web server

ซอฟต์แวร์บนเครื่อง server ที่รับ HTTP request และส่ง response

web service

back end ที่เปิดให้โปรแกรมอื่นใช้ ไม่ใช่ให้คนใช้ ตอบเป็นข้อมูล ไม่ใช่หน้าเว็บ